

Programming Assignment 01 A: Natural Language Processing with PySpark

Name: Devi Aditya Dantuluri

Text analytics on Jane Austen's *Pride and Prejudice* using PySpark DataFrame functions (`split` , `explode` , `lower` , `regexp_extract` , `regexp_replace` , `where`). Each cell is commented in line to answer the tasks in the assignment script.

```
In [1]: ##### Import Jane Austen's novel Pride and Prejudice #####
import urllib.request

# URL for Pride and Prejudice plain text on Project Gutenberg
url = "https://www.gutenberg.org/files/1342/1342-0.txt"
file_name = "pride_and_prejudice.txt"

# Download the file and save it locally under the custom file name.
try:
    urllib.request.urlretrieve(url, file_name)
    print("Downloaded from Project Gutenberg")
except Exception as e:
    # Fallback mirror of the same Gutenberg text (used if gutenberg.org is unreachable from this environmen
    mirror = "https://raw.githubusercontent.com/GITenberg/Pride-and-Prejudice_1342/master/1342-0.txt"
    urllib.request.urlretrieve(mirror, file_name)
    print("gutenberg.org unreachable, downloaded the same text from the GITenberg mirror")
```

gutenberg.org unreachable, downloaded the same text from the GITenberg mirror

```
In [2]: # Import SparkSession (the entry point to Spark) and the DataFrame functions used in this assignment.
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, split, explode, lower, regexp_extract, regexp_replace

# Create (or reuse) a SparkSession. All DataFrame operations go through this object.
spark = SparkSession.builder.getOrCreate()
spark.sparkContext.setLogLevel("ERROR")

# Read the text file into a DataFrame with one column called "value"; each row is one line of the book.
book = spark.read.text(file_name)

# Print the schema and the number of lines to understand what we loaded.
book.printSchema()
print("Number of lines in the file:", book.count())
```

```
root
|-- value: string (nullable = true)
```

Number of lines in the file: 13427

```
In [3]: # Display the first 200 rows (lines) of the book, exactly as they were read from the text file.
book.show(200)
```

```
+-----+
|          value|
+-----+
|The Project Guten...|
|This eBook is for...|
|almost no restric...|
|re-use it under t...|
|with this eBook o...|
|
|Title: Pride and ...|
|
|Author: Jane Austen|
|
|Posting Date: Aug...|
|Release Date: Jun...|
|Last Updated: Mar...|
```

| Language: English |
| Character set enc... |
| *** START OF THIS... |
|
| Produced by Anony... |
|
| PRIDE AND PREJUDICE |
| By Jane Austen |
|
| Chapter 1 |
|
| It is a truth uni... |
| of a good fortune... |
|
| However little kn... |
| first entering a ... |
| of the surroundin... |
| of some one or ot... |
|
| "My dear Mr. Benn... |
| Netherfield Park ... |
|
| Mr. Bennet replie... |
|
| "But it is," retu... |
| told me all about... |
|
| Mr. Bennet made n... |
|
| "Do you not want ... |
|
| "_You_ want to te... |
|
| This was invitati... |
|
| "Why, my dear, yo... |
| by a young man of... |
| down on Monday in... |
| delighted with it... |
| is to take posses... |
| be in the house b... |
|
| "What is his name?" |
|
| "Bingley." |
|
| "Is he married or... |
|
| "Oh! Single, my d... |
| five thousand a y... |
|
| "How so? How can ... |
|
| "My dear Mr. Benn... |
| must know that I ... |
|
| "Is that his desi... |
|
| "Design! Nonsense...

|_may_ fall in lov...|
| soon as he comes."|

|"I see no occasio...|
|them by themselve...|
|as handsome as an...|
| party."|

|"My dear, you fla...|
|I do not pretend ...|
|grown-up daughter...|

|"In such cases, a...|

|"But, my dear, yo...|
|the neighbourhood."|

|"It is more than ...|

|"But consider you...|
|be for one of the...|
|go, merely on tha...|
|newcomers. Indeed...|
|visit him if you ...|

|"You are over-scr...|
|glad to see you; ...|
|hearty consent to...|
|I must throw in a...|

|"I desire you wil...|
|others; and I am ...|
|good-humoured as ...|

|"They have none o...|
|all silly and ign...|
|quickness than he...|

|"Mr. Bennet, how ...|
|take delight in v...|

|"You mistake me, ...|
|are my old friend...|
|these last twenty...|

|"Ah, you do not k...|

|"But I hope you w...|
|thousand a year c...|

|"It will be no us...|
| visit them."|

|"Depend upon it, ...|
| all."|

|Mr. Bennet was so...|
|reserve, and capr...|
|been insufficient...|
|was less difficul...|
|little informatio...|
|she fancied herse...|
|daughters married...|

| Chapter 2|

|Mr. Bennet was am...|
|had always intend...|
|his wife that he ...|
|paid she had no k...|

```
|manner. Observing...|
|suddenly addresse...|
|
|"I hope Mr. Bingl...|
|
|"We are not in a ...|
|resentfully, "sin...|
|
|"But you forget, ...|
|assemblies, and t...|
|
|"I do not believe...|
|of her own. She i...|
|                of her."|
|
|"No more have I,"...|
|not depend on her...|
|
|Mrs. Bennet deign...|
|herself, began sc...|
|
|"Don't keep cough...|
|compassion on my ...|
|
|"Kitty has no dis...|
|                them ill."|
|
|"I do not cough f...|
|your next ball to...|
|
|"To-morrow fortni...|
|
|"Aye, so it is," ...|
|till the day befo...|
|for she will not ...|
|
|"Then, my dear, y...|
|Mr. Bingley to _h...|
|
+-----+
only showing top 200 rows
```

Task 1 and 2: `split`, `explode`, `lower` with `show(5)` and a one-sentence explanation above each line

```
In [4]: # split() breaks each line (string) into an array of words using a single space as the delimiter,
# and alias("line") names the resulting array column "line".
lines = book.select(split(book.value, " ").alias("line"))

# Show the first five rows after split: each row is now an array of strings instead of one string.
lines.show(5, truncate=False)
```

```
+-----+
|line                                         |
+-----+
|[The, Project, Gutenberg, EBook, of, Pride, and, Prejudice,, by, Jane, Austen]      |
|[]                                          |
|[This, eBook, is, for, the, use, of, anyone, anywhere, at, no, cost, and, with]      |
|[almost, no, restrictions, whatsoever., , You, may, copy, it,, give, it, away, or]    |
|[re-use, it, under, the, terms, of, the, Project, Gutenberg, License, included]      |
+-----+
only showing top 5 rows
```

```
In [5]: # The original script calls lines.show(1000); a shorter view is shown here to keep the exported PDF readable
# (Change 20 back to 1000 to reproduce the original output.)
lines.show(20)
```

```
+-----+
|          line|
+-----+
|[The, Project, Gu...|
|          []|
|[This, eBook, is,...|
|[almost, no, rest...|
|[re-use, it, unde...|
|[with, this, eBoo...|
|          []|
|          []|
|[Title:, Pride, a...|
|          []|
|[Author:, Jane, A...|
|          []|
|[Posting, Date:, ...|
|[Release, Date:, ...|
|[Last, Updated:, ...|
|          []|
|[Language:, English]|
|          []|
|[Character, set, ...|
|          []|
+-----+
only showing top 20 rows
```

```
In [6]: # explode() takes an array column and produces one row per element, so the array of words in
# each line becomes many rows with a single word each; alias("word") names that column "word".
words = lines.select(explode(col("line")).alias("word"))

# Show the first five rows after explode: one word per row.
words.show(5)
```

```
+-----+
|      word|
+-----+
|      The|
|    Project|
|Gutenberg|
|      eBook|
|        of|
+-----+
only showing top 5 rows
```

```
In [7]: # lower() converts every word to lowercase so that "The", "THE", and "the" are treated as the same token.
words_lower = words.select(lower(col("word")).alias("word_lower"))

# Show the first five rows after lower.
words_lower.show(5)
```

```
+-----+
|word_lower|
+-----+
|      the|
|    project|
|gutenberg|
|      ebook|
|        of|
+-----+
only showing top 5 rows
```

Task 3: what does the regular expression `[a-z]*` do?

`regexp_extract(col, pattern, 0)` returns the **first substring** of the column value that matches `pattern` (group 0 = the whole match).

- `[a-z]` is a character class that matches exactly one lowercase letter from a to z.
- `*` means "zero or more repetitions" of the preceding item.
- So `[a-z]*` matches the **longest run of consecutive lowercase letters starting at the first position where the engine finds a match**. Because `*` allows zero repetitions, the pattern always matches, and it matches at position 0 of the string.

Effect on individual words (after `lower()` has been applied):

input word	[a-z]* extracts	why
elizabeth	elizabeth	whole word is letters
bennet.	bennet	stops at the period
"mr.	`` (empty)	first character " is not a letter, so the match at position 0 is empty
1813	`` (empty)	digits are not in [a-z]
don't	don	stops at the apostrophe

In other words it strips **trailing** punctuation/digits from words, but any word that *begins* with a non-letter becomes an empty string (which is why the script filters empty strings afterwards). It does not remove punctuation that appears in the middle of a word; it just cuts the word at that point.

```
In [8]: # regexp_extract() returns the first substring of "word_lower" that matches the pattern "[a-z]*"
# (group index 0 = the entire match), i.e. the leading run of lowercase letters; alias("word") names it "word"
words_clean = words_lower.select(
    regexp_extract(col("word_lower"), "[a-z]*", 0).alias("word")
)

# Show the first five cleaned words.
words_clean.show(5)
```

```
+-----+
|   word|
+-----+
|   the |
| project|
|guttenberg|
|   ebook|
|     of |
+-----+
```

only showing top 5 rows

```
In [9]: # Demonstrate "[a-z]*" on a few hand-picked tokens to make its behaviour explicit.
demo = spark.createDataFrame(
    [("elizabeth"), ("bennet."), ("mr."), ("1813"), ("don't"), ("pride-and-prejudice"),],
    ["word_lower"],
)
demo.select(
    col("word_lower"),
    regexp_extract(col("word_lower"), "[a-z]*", 0).alias("extracted_with_[a-z]*"),
).show(truncate=False)
```

```
+-----+-----+
|word_lower|extracted_with_[a-z]*|
+-----+-----+
|elizabeth|elizabeth|
|bennet.|bennet|
|"mr.|
|1813|
|don't|don|
|pride-and-prejudice|pride|
+-----+-----+
```

Task 4: two more regular expressions for cleaning words

`regexp_replace(col, pattern, replacement)` replaces **every** substring that matches `pattern` with `replacement`. Using an empty replacement deletes the matched characters.

1. **Remove all numeric digits:** pattern `[0-9]` (equivalently `\d`) matches a single digit; replacing every digit with `""` removes them all. `Sama86d -> Samad`, `Samad0098 -> Samad`.
2. **Keep only alphabetic letters:** pattern `[^a-zA-Z]` matches any character that is **not** a letter (`^` inside the brackets negates the class); replacing every such character with `""` leaves only letters. `##Samad -> Samad`, `$Samad$ -> Samad`.

Both are tested below with the instructor's examples and with my own name.

```
In [10]: # Build a small test DataFrame with the example tokens from the assignment plus my own name.
test_words = spark.createDataFrame(
    [
        ("Sama86d",), ("Samad0098",), ("%Samad",), ("Samad$",),
        ("Adi86tya",), ("Aditya0098",), ("%Aditya",), ("Aditya$",), ("Dantulur!i_2026",)],
    ["word"],
)

# Expression 1: "[0-9]" matches any single digit; regexp_replace deletes every digit in the word.
# Expression 2: "[^a-zA-Z]" matches any character that is NOT a letter; regexp_replace deletes every such c
test_words.select(
    col("word"),
    regexp_replace(col("word"), "[0-9]", "").alias("digits_removed"),
    regexp_replace(col("word"), "[^a-zA-Z]", "").alias("letters_only"),
).show(truncate=False)
```

word	digits_removed	letters_only
Sama86d	Samad	Samad
Samad0098	Samad	Samad
%Samad	%Samad	Samad
\$Samad\$	\$Samad\$	Samad
Adi86tya	Aditya	Aditya
Aditya0098	Aditya	Aditya
%Aditya	%Aditya	Aditya
\$Aditya\$	\$Aditya\$	Aditya
Dantulur!i_2026	Dantuuri	Dantuuri

```
In [11]: # Apply the two new cleaning expressions to the actual book words (after lower()), for comparison
# with the original "[a-z]*" extraction. Distinct values shown so the differences are visible.
comparison = words_lower.select(
    col("word_lower"),
    regexp_extract(col("word_lower"), "[a-z]*", 0).alias("extract_[a-z]*"),
    regexp_replace(col("word_lower"), "[0-9]", "").alias("digits_removed"),
    regexp_replace(col("word_lower"), "[^a-z]", "").alias("letters_only"),
)

# Only rows where the three methods disagree, to show where each expression behaves differently.
comparison.where(
    (col("extract_[a-z]*") != col("letters_only")) | (col("digits_removed") != col("word_lower"))
).distinct().show(20, truncate=False)
```

word_lower	extract_[a-z]*	digits_removed	letters_only
ma'am?--is	ma	ma'am?--is	maamis
"caroline		"caroline	caroline
well-wisher	well	well-wisher	wellwisher
speaking--stating	speaking	speaking--stating	speakingstating
is!--her	is	is!--her	isher
34			
"implacable		"implacable	implacable
atonement--for	atonement	atonement--for	atonementfor
dishonest--for	dishonest	dishonest--for	dishonestfor
"sir		"sir	sir
satisfied--and	satisfied	satisfied--and	satisfiedand
slowly--and	slowly	slowly--and	slowlyand
you.		_you_.	you
can		_can_	can
"by		"by	by
mother's	mother	mother's	mothers
"tell		"tell	tell
parsonage-house,	parsonage	parsonage-house,	parsonagehouse
good-will.	good	good-will.	goodwill
him.		_him_.	him

only showing top 20 rows

Task 5: role of the `where()` function

`where()` (an alias of `filter()`) **keeps only the rows for which a Boolean condition is true** and drops all other rows. It does not change columns; it changes which rows survive. In the script it is used as `words_clean.where(col("word") != "")` to throw away the empty strings that `regex_extract` produced for tokens that started with a non-letter (punctuation, digits, blank lines). It is the DataFrame equivalent of a SQL `WHERE` clause.

```
In [12]: # where() keeps only rows whose "word" column is not the empty string, removing the blanks created by regex
words_nonull = words_clean.where(col("word") != "")

# Compare the row counts before and after filtering to see how many empty tokens were removed.
print("Rows before where():", words_clean.count())
print("Rows after where():", words_nonull.count())
words_nonull.show(5)
```

Rows before where(): 127368

Rows after where(): 122157

+-----+

| word |

+-----+

| the |

| project |

| gutenberg |

| ebook |

| of |

+-----+

only showing top 5 rows

Task 6: remove the word "the" with `where()`

```
In [13]: # Count how many times "the" occurs before removal (it is the most frequent word in the novel).
print('Occurrences of "the" before removal:', words_nonull.where(col("word") == "the").count())

# where() with the condition col("word") != "the" keeps every row except those whose word is exactly "the".
words_without_the = words_nonull.where(col("word") != "the")

# Verify: the count of "the" should now be zero, and the total row count drops by the same amount.
print('Occurrences of "the" after removal :', words_without_the.where(col("word") == "the").count())
print("Total words before:", words_nonull.count(), "| after:", words_without_the.count())
words_without_the.show(10)
```

Occurrences of "the" before removal: 4480

Occurrences of "the" after removal : 0

Total words before: 122157 | after: 117677

+-----+

| word |

+-----+

| project |

| gutenberg |

| ebook |

| of |

| pride |

| and |

| prejudice |

| by |

| jane |

| austen |

+-----+

only showing top 10 rows

```
In [14]: # Extra check: the top 10 most frequent words before and after removing "the".
print("Top 10 words BEFORE removing 'the':")
words_nonull.groupBy("word").count().orderBy(col("count").desc()).show(10)
print("Top 10 words AFTER removing 'the':")
words_without_the.groupBy("word").count().orderBy(col("count").desc()).show(10)
```

Top 10 words BEFORE removing 'the':

```

+----+-----+
|word|count|
+----+-----+
| the| 4480|
|  to| 4218|
|  of| 3711|
| and| 3504|
| her| 2199|
|   a| 1983|
|  in| 1909|
| was| 1838|
|   i| 1750|
| she| 1668|
+----+-----+
only showing top 10 rows
Top 10 words AFTER removing 'the':
+----+-----+
|word|count|
+----+-----+
|  to| 4218|
|  of| 3711|
| and| 3504|
| her| 2199|
|   a| 1983|
|  in| 1909|
| was| 1838|
|   i| 1750|
| she| 1668|
|that| 1487|
+----+-----+
only showing top 10 rows

```

```

In [15]: # Stop the SparkSession to release resources.
         spark.stop()

```